

StatsPAI: A Unified, Agent-Native Python Toolkit for Causal Inference and Applied Econometrics

Biaoyue Wang^{1,2} and Scott Rozelle^{1,2}

¹ Rural Education Action Program, Stanford Center on China's Economy and Institutions, Stanford University, United States ² StatsPAI Inc., United States Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- Review
- Repository
- Archive

Editor: Richard Liu

Reviewers:

- @Carol-seven
- @FATelarico
- @MartinSpindler

Submitted: 07 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

StatsPAI is an open-source Python package for causal inference and applied econometrics. It gives empirical researchers a single interface for estimating, diagnosing, comparing, and reporting models that are usually spread across many specialized packages or proprietary statistical environments. A single import `statspai as sp` reaches estimators for the main families of applied work — regression and panel models, instrumental variables, the modern difference-in-differences and regression-discontinuity toolkits, synthetic control and matching, and machine-learning estimators of heterogeneous treatment effects — together with the diagnostics, robustness checks, and reporting that surround them. The full catalogue of more than 1,000 registered functions across 81 submodules is enumerated in the package documentation rather than here.

The package is designed for policy evaluation, social science and public health research, and other empirical workflows where researchers must move between research design, estimation, diagnostics, robustness checks, and publication tables. Results from the mature estimators share a common reporting surface, so the same calls produce a summary, a figure, a LaTeX or Word table, or a citation across the estimators that support them.

StatsPAI is also agent-native: every registered function exposes a machine-readable schema — a structured description of its arguments and outputs that a program can parse directly — together with structured failure metadata. This lets LLM-driven research assistants discover estimators, choose among alternatives, and surface the assumptions behind a method without parsing free-form prose, the capability that most distinguishes StatsPAI from a conventional estimator library. The source code is available at <https://github.com/brycewang-stanford/StatsPAI> and archived on Zenodo (Wang & Rozelle, 2026).

Statement of Need

Applied researchers face a fragmented software landscape. Stata offers an integrated workflow, but it is proprietary and does not expose a typed, machine-readable interface for AI-assisted analysis. R provides excellent method-specific packages such as `did` (Callaway & Sant'Anna, 2021), `rdrobust` (Calonico et al., 2014), `Synth` (Abadie et al., 2010), `grf` (Athey et al., 2019), and `lme4` (Bates et al., 2015), but these packages use different APIs, object systems, output conventions, and diagnostic workflows. Python has strong pieces of the causal inference ecosystem, including DoWhy for graphical causal models (Sharma & Kiciman, 2020), EconML for machine-learning treatment effect estimation (Battocchi et al., 2019), CausalML for uplift modeling (Chen et al., 2020), and DoubleML for double/debiased machine learning (Bach et al., 2022). None of these tools, however, is intended to cover the full applied-econometrics workflow from design diagnosis through estimation, robustness, and publication output.

StatsPAI addresses this gap for graduate students, applied economists, policy researchers, and data scientists who want a Python-native workflow without giving up the breadth of Stata or the methodological depth of R. The goal of StatsPAI is not to replace every specialized implementation. Instead, it provides a coherent empirical workspace: shared formula conventions, compatible result surfaces for mature estimators, export methods where supported, citations attached to estimators, and validation metadata that make the relationship between methods, assumptions, and evidence explicit.

State of the Field

Existing Python packages are strongest when they focus on a narrower problem. DoWhy emphasizes identification, graphical assumptions, and refutation; EconML and CausalML focus on heterogeneous effects and uplift modeling; DoubleML implements orthogonal-score estimators for a well-defined family of double machine-learning designs. These packages are complementary to StatsPAI, and several ideas in StatsPAI follow the same methodological literature, including double/debiased machine learning (Chernozhukov et al., 2018), causal forests (Wager & Athey, 2018), and meta-learners (Künzel et al., 2019). These comparisons are with packages that, like StatsPAI, operate at the causal-inference and machine-learning layer. The general-purpose regression toolkits applied economists already rely on — statsmodels and linearmodels in Python, or fixest in R — sit at a different layer: they supply core regression machinery, not the integrated workflow spanning design, estimation, diagnostics, and reporting that StatsPAI targets. StatsPAI depends on that stack — statsmodels and linearmodels are part of its own foundation — while implementing the causal-inference estimators it specializes in and, where an established reference implementation exists, validating them for numerical agreement against the corresponding R and Stata packages. A direct head-to-head is therefore not the relevant test; the question is whether the integration layer StatsPAI adds is worth maintaining as a separate package.

The build-versus-contribute case for StatsPAI is therefore about scope and interface rather than a single estimator. Contributing one more estimator to each existing project would still leave users with many incompatible result classes, separate diagnostic conventions, and no unified agent-facing schema. StatsPAI contributes an integration layer with substantive statistical content: broad method coverage, shared reporting, explicit estimator citations, stable/experimental API metadata, cross-language parity checks, and an LLM-oriented registry that can expose statistical tools safely to automated workflows. StatsPAI therefore earns a standalone existence not by contributing a better single estimator, but by being the layer that makes a heterogeneous collection of estimators — classical and machine-learning, Python-native and R/Stata-aligned — behave as one coherent, agent-addressable workspace. Folding that layer into any one upstream project would not serve the researchers who must move across all of them.

Software Design

StatsPAI is organized around method families and a registry layer. Researchers can call focused functions, such as an IV or regression-discontinuity estimator, or use higher-level dispatchers that select among variants within a design family. The registry records function names, parameters, examples, stability tiers, limitations, citations, and schema information. This makes the package usable both from a notebook and from external systems such as a Model Context Protocol server. In practice this lets an assistant query the registry for the estimators valid for a detected design, invoke one, read back structured diagnostics and assumption violations, and decide the next step — all through typed schemas rather than free-form prompts.

The central design choice is a shared result interface. Estimators return structured objects that

89 store coefficients, uncertainty estimates, diagnostics, fitted values, plots, and exporter hooks in
90 predictable locations. This reduces the switching cost between classical econometric estimators
91 and modern machine-learning estimators, and it also makes validation easier because tests can
92 compare common fields across implementations.

93 The package is implemented mainly in Python on top of NumPy, SciPy, Pandas, statsmodels,
94 scikit-learn, and linearmodels. This keeps the installation path familiar for Python users and
95 supports Python 3.9 and newer versions of Python. Optional accelerator backends are used only
96 where they materially change the computation: PyTorch for neural causal estimators, JAX for
97 selected bootstrap and linear algebra workloads, and a Rust/PyO3 kernel for high-dimensional
98 fixed-effect and cluster-variance routines. This keeps the default package inspectable while
99 allowing heavy workloads to use specialized backends when available. The package is distributed
100 via PyPI under the MIT license.

101 These choices carry costs worth stating plainly. A shared result interface means that adding or
102 upgrading an estimator is never purely local: each estimator must be mapped onto the common
103 fields, and the reporting and export hooks are guaranteed only for the mature estimators,
104 which is why auxiliary helpers advertise narrower capabilities through the registry rather than
105 claiming a uniformity they do not have. Favouring breadth across method families also trades
106 against depth: for any single design, a dedicated package may expose more edge-case options
107 or finer tuning than StatsPAI does today. Finally, performance is deliberately not the first
108 priority of the default install — the package runs on a pure-Python NumPy/SciPy stack
109 and reaches for PyTorch, JAX, or a Rust kernel only when they are present, falling back
110 transparently otherwise — which keeps the default small and inspectable at the cost of leaving
111 peak performance to an optional, environment-dependent layer whose accelerated paths are
112 held to the same documented numerical tolerances as the fallbacks. The Rust/PyO3 kernel in
113 particular adds a compiled-language component to the build: shipping it requires platform-
114 specific wheels (or a Rust toolchain to compile from source) and a separately maintained crate,
115 a maintenance cost we contain by keeping the kernel optional and transparently falling back
116 to the pure-Python implementation wherever the compiled extension is absent. We accept
117 these costs deliberately: for researchers who must compare estimators, switch designs, and
118 produce reproducible output within one project, a single coherent, agent-addressable interface
119 outweighs the loss of per-method specialization.

120 Research Impact Statement

121 StatsPAI ships a concrete validation and community-readiness dossier built from two
122 complementary tracks. The first is a cross-language parity harness: StatsPAI, R, and Stata
123 are run on the same input data and their numerical output is compared directly. The harness
124 checks more than sixty estimator modules against a reference R implementation on identical
125 input bytes, the large majority of which are also checked against Stata. On closed-form
126 estimators the three languages agree to machine precision; iterative and machine-learning
127 estimators agree within pre-registered, documented tolerances, and the few remaining
128 convention gaps are disclosed rather than hidden. The second track calibrates the simulated
129 teaching datasets bundled in `sp.datasets` so that the canonical estimator recovers values in
130 the neighbourhood of well-known published results: returns-to-schooling IV (Card), job-training
131 effects (LaLonde/Dehejia-Wahba), regression-discontinuity elections (Lee), multi-period
132 difference-in-differences (Callaway-Sant'Anna), and synthetic control. Because these datasets
133 are simulated rather than the original study data, exact numerical replication is deliberately not
134 claimed. The validation suite also includes a 1000-replication coverage run for representative
135 OLS, difference-in-differences, and strong-instrument IV designs, with empirical coverage close
136 to the nominal 95 percent level. A reviewer-facing validation dossier and a short reviewer
137 guide are included in the repository documentation.

138 The near-term research impact is a more reproducible empirical workflow for applied policy
139 evaluation. Because methods share one interface, researchers can compare estimators on the

140 same data, export tables with the same metadata, and record the citations and assumptions
141 attached to each analysis. StatsPAI is currently being used in an ongoing working paper
142 connected to the Rural Education Action Program at Stanford University, *Family contagion*
143 *of screen time? Within-person evidence from six waves in China* (Wang, Zhang, and Hou,
144 in preparation), which relies on the package for its panel and within-person estimation; no
145 peer-reviewed research article using the package has yet been published. At this stage, then,
146 the impact claim rests on three things a reviewer can check directly: active use in an ongoing
147 working paper, public distribution on PyPI, and the reproducible validation materials and worked
148 examples bundled with the repository. The agent-native registry also supports AI-assisted
149 replication and robustness analysis in which statistical tools are discovered and invoked through
150 explicit schemas rather than informal prompts.

151 AI Usage Disclosure

152 Generative AI tools, including Claude Code and OpenAI ChatGPT/Codex, were used for code-
153 generation assistance, refactoring suggestions, test scaffolding, documentation drafting, and
154 manuscript copy-editing. Exact model identifiers were not retained for all exploratory sessions.
155 Human authors made the core design decisions; reviewed, edited, and checked AI-assisted code
156 and prose; and checked citations and software claims against repository evidence. The authors
157 will not use generative AI to produce substantive responses to journal editors or reviewers.
158 All authors take responsibility for the correctness, originality, licensing, and compliance of the
159 package and this paper.

160 Author Contributions

161 **Biaoyue Wang** conceived and designed the package, implemented the estimators, registry,
162 schema layer, and result objects, wrote the documentation, tests, and validation suites, and led
163 the drafting of this paper. **Scott Rozelle** provided guidance on the package's design direction
164 and target research workflows, and contributed to the writing, review, and revision of this
165 paper. Both authors reviewed and approved the final manuscript and take responsibility for
166 the correctness of the package and this paper.

167 Acknowledgements

168 The authors thank the Stanford Rural Education Action Program (REAP) research community
169 and the CoPaper.AI team for feedback on early workflows. StatsPAI Inc. is the legal entity
170 associated with the project, and CoPaper.AI is a commercial downstream product that may
171 call the MIT-licensed StatsPAI package; the StatsPAI package itself is permanently open
172 source under the MIT license. The authors are also grateful to the developers of NumPy, SciPy,
173 Pandas, statsmodels, scikit-learn, linearmodels, PyTorch, JAX, and the broader open-source
174 scientific Python ecosystem that StatsPAI builds upon.

175 References

- 176 Abadie, A., Diamond, A., & Hainmueller, J. (2010). Synthetic control methods for comparative
177 case studies: Estimating the effect of California's tobacco control program. *Journal of the*
178 *American Statistical Association*, 105(490), 493–505. [https://doi.org/10.1198/jasa.2009.](https://doi.org/10.1198/jasa.2009.ap08746)
179 [ap08746](https://doi.org/10.1198/jasa.2009.ap08746)
- 180 Athey, S., Tibshirani, J., & Wager, S. (2019). Generalized random forests. *The Annals of*
181 *Statistics*, 47(2), 1148–1178. <https://doi.org/10.1214/18-aos1709>

- 182 Bach, P., Chernozhukov, V., Kurz, M. S., & Spindler, M. (2022). DoubleML – an object-
183 oriented implementation of double machine learning in Python. *Journal of Machine Learning*
184 *Research*, 23(53), 1–6. <https://jmlr.org/papers/v23/21-0862.html>
- 185 Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models
186 using lme4. *Journal of Statistical Software*, 67(1), 1–48. <https://doi.org/10.18637/jss.v067.i01>
- 188 Battocchi, K., Dillon, E., Hei, M., Lewis, G., Oka, P., Oprescu, M., & Syrgkanis, V. (2019).
189 *EconML: A Python package for ML-based heterogeneous treatment effects estimation*.
190 <https://github.com/py-why/EconML>
- 191 Callaway, B., & Sant’Anna, P. H. C. (2021). Difference-in-differences with multiple time periods.
192 *Journal of Econometrics*, 225(2), 200–230. <https://doi.org/10.1016/j.jeconom.2020.12.001>
- 193 Calonico, S., Cattaneo, M. D., & Titiunik, R. (2014). Robust nonparametric confidence
194 intervals for regression-discontinuity designs. *Econometrica*, 82(6), 2295–2326. <https://doi.org/10.3982/ECTA11757>
- 196 Chen, H., Harinen, T., Lee, J.-Y., Yung, M., & Zhao, Z. (2020). *CausalML: Python package*
197 *for causal machine learning*. <https://doi.org/10.48550/arXiv.2002.11631>
- 198 Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., & Robins,
199 J. (2018). Double/debiased machine learning for treatment and structural parameters. *The*
200 *Econometrics Journal*, 21(1), C1–C68. <https://doi.org/10.1111/ectj.12097>
- 201 Künzel, S. R., Sekhon, J. S., Bickel, P. J., & Yu, B. (2019). Metalearners for estimating
202 heterogeneous treatment effects using machine learning. *Proceedings of the National*
203 *Academy of Sciences*, 116(10), 4156–4165. <https://doi.org/10.1073/pnas.1804597116>
- 204 Sharma, A., & Kiciman, E. (2020). *DoWhy: An end-to-end library for causal inference*.
205 <https://doi.org/10.48550/arXiv.2011.04216>
- 206 Wager, S., & Athey, S. (2018). Estimation and inference of heterogeneous treatment effects
207 using random forests. *Journal of the American Statistical Association*, 113(523), 1228–1242.
208 <https://doi.org/10.1080/01621459.2017.1319839>
- 209 Wang, B., & Rozelle, S. (2026). *StatsPAI: The agent-native causal inference and econometrics*
210 *toolkit for python* (Version 1.15.6). <https://doi.org/10.5281/zenodo.19933900>